

ADR 0001 — Plataforma de backend do BdayList

- **Status:** Aceito
- **Data:** 2026-06-23
- **Decisores:** time BdayList (arq, back, dba, redteam)
- **Contexto de produto:** lista de presentes de aniversário (estilo casar.com). SPA Next.js 16 com **static export**, deploy Netlify. SEM dinheiro no escopo atual ("presentear" = reservar).
- **Estágio:** MVP bootstrapped (custo/operação mínimos, ir pro ar rápido sem comprometer integridade).

Este ADR também compara a hipótese de um **backend dedicado/separado**, a pedido, com análise baseada em dados (preços/limites consultados em 2026-06-23). Ver datas e fontes ao final.

1. Decisão

Adotar **(A) SPA static export + Supabase Cloud** na região **sa-east-1** (São Paulo), com **Postgres + RLS + Edge Functions + pg_cron**, e-mail transacional via Resend.

Onde mora cada responsabilidade:

Responsabilidade	Onde roda
Leitura da lista pública (por token), RSVP, painel do host	Cliente <code>supabase-js</code> sob RLS / RPC SECURITY DEFINER
Reserva de presente (atômica, idempotente, anti-TOCTOU)	Postgres function (SECURITY DEFINER) via RPC/Edge Function
Privacidade "surpresa", token de compartilhamento	RLS + função no banco
E-mail, enriquecimento de link (anti-SSRF), jobs	Edge Functions (<code>service_role</code>) + <code>pg_cron</code>

Banco: **PostgreSQL** (decisão detalhada no **ADR 0002**).

2. Opções consideradas

Opção	Resumo
(A) SPA + BaaS (Supabase Cloud) ✓	Browser fala direto com Postgres+Auth via RLS; lógica sensível em Postgres functions / Edge Functions. Zero backend para operar.
(B) SPA + Postgres gerenciado (Neon) + backend dedicado	A SPA chama uma API HTTP própria (Hono/Fastify/NestJS em Render/Fly/Railway); toda authz numa camada de serviço; banco fechado.
(C) Next.js com servidor (SSR/Route Handlers) + Postgres	Abandona o static export; deploy passa a exigir runtime Node. Custo de mudança alto sobre o que já existe.
(D) Self-host (Supabase self-hosted ou Postgres em VPS)	Mais barato em compute bruto; caro em tempo humano (patching, backup, segurança, scaling manuais).

3. Comparativo baseado em dados (2026-06-23)

Critério	(A) Supabase	(B) Neon + backend	(C) Next server + Neon	(D) Self-host VPS
Custo MVP (free)	Free: 500 MB DB, 1 GB storage, 50k MAU; pausa após 7d inativo	Neon Free + Render Free (cold start) ou Railway ~US\$5	Sem free real (runtime sempre on)	VPS ~US\$4–6/mês
1º tier pago real	Pro US\$25/mês (+overage)	~US\$10–12/mês (Neon Launch US\$5 + Render/Railway US\$5–7)	idem (B) + host	US\$4–12 + seu tempo
Esforço de operação	Mínimo	Médio-alto	Médio-alto	Alto
Cold start / latência	API sem cold start	Render Free dorme; Fly/Railway always-on ok	possível cold start	sem cold start
Latência BR	região São Paulo	Neon SP + backend GRU	idem	VPS BR
Vendor lock-in	Médio (mas é Postgres puro embaixo → exportável)	Baixo	Baixo-médio	Baixo
Fit com static export	Excelente	Bom	Rompe o static export	Bom
Time-to-market	Mais rápido	Lento	Lento	Mais lento

Notas:

- **Free tier do Supabase pausa após 7 dias de inatividade** — irrelevante em produção com tráfego, mas crítico em pré-lançamento (lista que "some"). Mitigação: subir para Pro antes do lançamento público.
- **PITR no Supabase é add-on caro (~US\$100/mês)** — no MVP, viver com backup diário do Pro e ligar PITR só quando houver dado de valor irreversível.

4. BaaS vs backend dedicado — o trade-off central

A decisão real é **onde mora a autorização**.

Dimensão	BaaS (RLS no Postgres)	Backend dedicado (camada de serviço)
Authz	Declarativa, no banco, fail-closed (sem policy = sem linha). Vale mesmo com a chave publishable pública.	Imperativa, em código. Flexível, mas um endpoint esquecido = vazamento.
Reserva atômica	<code>UPDATE ... WHERE status='disponivel'</code> + índice único parcial — atômico e idempotente nativamente.	Mesma garantia, mais boilerplate (pool, transação, retry).
Privacidade "surpresa"	RLS + VIEW que omite quem reservou.	Filtro no serializer; risco de vazár em endpoint novo.
Complexidade	Baixa: banco + policies + RPC.	Alta: API, auth, conexões, deploy, observabilidade.
Evolução	Rápida no CRUD; atrito quando a regra não cabe em SQL/RLS.	Lenta no início; confortável para lógica rica e integrações.
Risco principal	RLS esquecida numa tabela nova expõe tudo (disciplina operacional).	Middleware de authz esquecido num endpoint (erro de omissão).

Quando BaaS vale: domínio majoritariamente CRUD + poucas operações sensíveis isoláveis em RPC. Exatamente o BdayList.
Quando backend dedicado vale: lógica de negócio rica fora do alcance de RLS, múltiplas integrações orquestradas, API pública versionada para terceiros, ou regulatório que exige camada própria. **Nada disso é o caso hoje.**

Ponto frequentemente esquecido: **Supabase é Postgres puro**. O lock-in real é Auth + Realtime + Storage, não os dados — migrar o banco para Neon depois é um `pg_dump`. Isso derruba o argumento mais forte contra BaaS no MVP.

5. Consequências

Positivas

- US\$0 em validação → US\$25/mês previsível em produção inicial; operação ~zero.
- Integridade da reserva e privacidade resolvidas no banco (menos código, menos superfície de erro).
- LGPD com menos atrito: dados em São Paulo, sem transferência internacional.
- Migração futura é evolutiva (Postgres exportável), não rewrite.

Negativas / riscos aceitos

- Dependência de RLS bem escrita — exige **disciplina e teste** (ver checklist no [doc de segurança](#)).
- Free tier pausa por inatividade — subir para Pro antes do lançamento público.
- PITR é add-on caro — adiar até haver dado que justifique.

6. Gatilhos de reavaliação (quando migrar para B)

1. Regras de negócio que **deixem de caber em RLS/RPC** sem virar SQL ilegível.
2. Entrada de **dinheiro** no escopo (gateway/PIX) exigindo orquestração, webhooks idempotentes e reconciliação.
3. Custo: cruzar para o tier **Team (US\$599/mês)** sem precisar de compliance que o justifique — aí Neon + backend dedicado fica mais barato.
4. Necessidade de **API pública versionada** para terceiros/integradores.

7. Stack recomendada

Camada	Recomendado	Alternativa
Dados + Auth	Supabase Cloud (Postgres + RLS + Auth), sa-east-1	Neon + backend dedicado (gatilho)
Lógica sensível	Postgres functions (SECURITY DEFINER) + RLS	Edge Functions p/ o que não cabe em SQL
Server-side / integrações	Supabase Edge Functions (Deno, service_role)	Cloudflare Workers
Jobs / cron	pg_cron (lembretes, limpeza de reservas órfãs)	Edge Function agendada
E-mail transacional	Resend (free 3.000/mês)	Amazon SES (mais barato em escala)
Storage de imagens	Supabase Storage	Cloudflare R2 (egress zero ao crescer)

8. Qual modelo de IA usar quando um agent for solicitado

Decisão de operação dos agents deste repo (`.claude/agents/`):

Tipo de agent	Modelo	Porquê
Arquitetura/decisão profunda (<code>arq</code> , <code>dba</code> , <code>redteam</code>)	<code>opus</code> (Claude Opus 4.8, <code>claude-opus-4-8</code>)	Raciocínio de alto nível, tradeoffs e long-horizon — o Opus-tier mais capaz.
Implementação/execução (<code>back</code> , <code>front</code> , <code>scribe</code> , <code>bug</code> , <code>qa</code> , <code>pixel</code>)	<code>sonnet</code> (Claude Sonnet 4.6)	Melhor equilíbrio velocidade/custo/inteligência para execução.
Tarefas mecânicas/baratas / subagents	<code>haiku</code> (Claude Haiku 4.5)	Rápido e econômico quando a tarefa não é sensível à inteligência.

O agent `dba` foi criado com `model: opus` por isso. Para trabalho de codificação/agentíc é recomendado rodar em effort `high` / `xhigh` . IDs canônicos: Opus 4.8 = `claude-opus-4-8` , Sonnet 4.6 = `claude-sonnet-4-6` , Haiku 4.5 = `claude-haiku-4-5` (fonte: skill `claude-api` , cache 2026-06-04).

Fontes (consultadas em 2026-06-23)

- Supabase: `pricing` · `regiões`
- Neon: `pricing`
- Render/Fly/Railway: `free tier 2026`
- Resend: `pricing` · Cloudflare R2: `pricing`